

Rešavanje problema raspoređivanja poslova po mašinama metaheuristikama

Minimum Job Shop Scheduling

Nenad Pešić

septembar 2026.

Sadržaj

1	Uvod	2
2	Opis rešenja	3
2.1	Kodiranje rešenja	3
2.1.1	Prirodan zapis i njegov nedostatak	3
2.1.2	Permutacija sa ponavljanjem	4
2.1.3	Višeznačnost zapisa	4
2.2	Dekoder	4
2.2.1	Postupak	5
2.2.2	Poluaktivni rasporedi	5
2.2.3	Složenost	5
2.2.4	Primer	5
3	Eksperimentalni rezultati	7
3.1	Okruženje za eksperimentisanje	7
3.2	Test instance	7
3.3	Metodologija	8
3.3.1	Budžet umesto vremena	8
3.3.2	Broj ponavljanja	9
3.3.3	Stohastičnost i reproducibilnost	9
3.4	Rezultati testiranja	10
3.5	Egzaktno rešavanje	11
3.6	Poređenje sa rezultatima iz literature	13
3.7	Diskusija	14
3.7.1	Uža okolina ne donosi bolji rezultat	14
3.7.2	Složenija metoda nije nužno bolja	16
3.7.3	Odnos prema egzaktnom rešavanju	16
4	Zaključak	17
5	Literatura	18

Poglavlje 1

Uvod

Poglavlje 2

Opis rešenja

2.1 Kodiranje rešenja

Za početak potrebno je formalno definisati rešenje ka kojem stremimo i odrediti format pogodan za primenu optimizacija koje slede. U ovom poglavlju obrađeno je upravo to pitanje, ali i sve pomoćne strukture neophodne za uspešno implementiranje optimizacija. Dati su opisi oblika okoline, načina ukrštanja, načina evaluacije i drugi.

2.1.1 Prirodan zapis i njegov nedostatak

Redosled operacija unutar posla zadat je instancom i ne može se menjati. Jedina dimenzija slobode koju rešavač poseduje jeste određivanje **redosleda koji operacije zauzimaju na svakoj mašini**. Nameće se, dakle, prirodan zapis, kao m permutacija, po jedan raspored za svaku mašinu.

Takav zapis ima ozbiljan nedostatak koji se vrlo brzo prikazuje. Naime, većina kombinacija ne odgovara nijednom validnom rasporedu. Posmatrajmo instancu sa dva posla i dve mašine:

$$J_1 : M_1(3) \rightarrow M_2(2), \quad J_2 : M_2(2) \rightarrow M_1(4)$$

Mašina M_1 obrađuje prvu operaciju posla J_1 i drugu operaciju posla J_2 , dok mašina M_2 obrađuje prvu operaciju posla J_2 i drugu operaciju posla J_1 . Za svaku mašinu postoje dva moguća redosleda, dakle ukupno četiri kombinacije:

Tabela 2.1: Sve kombinacije redosleda po mašinama za instancu sa dva posla i dve mašine.

redosled na M_1	redosled na M_2	C_{max}
J_1 pa J_2	J_2 pa J_1	7
J_1 pa J_2	J_1 pa J_2	11
J_2 pa J_1	J_2 pa J_1	11
J_2 pa J_1	J_1 pa J_2	—

Poslednja kombinacija **ne opisuje validan raspored**. Prva operacija posla J_1 čeka da se na mašini M_1 završi druga operacija posla J_2 . Ta operacija čeka prvu operaciju istog posla. Dalje, sad ona čeka da se na mašini M_2 završi druga operacija posla J_1 , a ta operacija čeka prvu

operaciju posla J_1 , od koje smo i pošli. Zavisnosti čine ciklus i nijedna operacija ne može da počne.

Na ovako maloj instanci neizvodljiva je jedna od četiri kombinacije. Sa porastom broja poslova i mašina taj udeo raste, pa bi svaka metoda koja slučajno bira redoslede po mašinama najveći deo rada trošila na odbacivanje neispravnih rešenja.

2.1.2 Permutacija sa ponavljanjem

Zbog toga je usvojeno kodiranje koje je za ovaj problem predložio Kristijan Birvirt (Bierwirth 1995). Rešenje je **lista oznaka poslova**, u kome se oznaka svakog posla javlja onoliko puta koliko taj posao ima operacija. Za instancu sa dva posla po dve operacije, jedan takav niz je

$$(J_2, J_1, J_1, J_2).$$

Niz se čita sleva nadesno, a k -to pojavljivanje oznake posla J_j predstavlja **k -tu operaciju** tog posla. Gornji niz stoga redom označava prvu operaciju posla J_2 , prvu i drugu operaciju posla J_1 , pa drugu operaciju posla J_2 .

Ključna osobina ovog zapisa jeste da je **svaki niz sa ispravnim brojem pojavljivanja izvodljiv**. $(k+1)$. operacija nekog posla ne može se pojaviti pre operacije k , jer je njeno mesto u nizu po definiciji kasnije pojavljivanje iste oznake. Redosled unutar posla time nije ograničenje koje se proverava, već osobina samog zapisa, pa ciklus opisan u prethodnom odeljku nije moguće ni zapisati u Birvirtovoj notaciji.

Broj različitih nizova za instancu sa poslovima $J_1, \dots, J_n$ iznosi

$$\frac{(\sum_j |J_j|)!}{\prod_j |J_j|!},$$

što za instancu **ft06** sa 36 operacija daje približno $2,67 \cdot 10^{24}$ različitih nizova.

2.1.3 Višeznačnost zapisa

Jedan od izazova ove notacije jeste to što ne garantuje uzajamnu jednoznačnost zapisa i rasporeda. Odnosno, više različitih nizova može dati isti raspored. Za pomenutu instancu sa dva posla postoji šest različitih nizova, koji se preslikavaju u svega tri izvodljiva rasporeda iz tabele 2.1. Čak četiri od šest nizova daju optimalno rešenje.

Prostor pretrage je, dakle, veći od prostora rešenja. Ipak, u literaturi je cena koja se plaća za garantovanu izvodljivost smatrana prihvatljivom. Navodi se (a što nije ovim radom eksplicitno proveravano) da je provera izvodljivosti u alternativnom zapisu znatno skuplja od višestrukog obilaska istog rasporeda.

2.2 Dekoder

Niz oznaka poslova sam po sebi nije raspored — on samo utvrđuje **redosled** kojim se operacije razmatraju. Postupak koji od tog niza pravi raspored, dakle dodeljuje svakoj operaciji trenutak početka, nazivamo *dekoderom*.

2.2.1 Postupak

Operacije se obrađuju redom kojim se javljaju u nizu. Za svaku od njih trenutak početka određen je sa dva ograničenja: operacija ne može početi pre nego što se završi prethodna operacija istog posla, niti pre nego što se oslobodi mašina koju zahteva. Najraniji trenutak koji zadovoljava oba jeste

$$s_o = \max(r_{J(o)}, f_{M(o)}),$$

gde je $r_{J(o)}$ trenutak završetka prethodne operacije posla kome o pripada, a $f_{M(o)}$ trenutak u kome se oslobađa mašina $M(o)$. Nakon dodele, obe vrednosti ažuriraju se na $s_o + p_o$.

Dekoder time održava tri niza vrednosti: vreme spremnosti svakog posla, vreme oslobađanja svake mašine i redni broj sledeće operacije svakog posla. Poslednji od njih pretvara oznaku posla u konkretnu operaciju, u skladu sa pravilom o k -tom pojavljivanju.

Vrednost funkcije cilja, C_{max} , jednaka je najvećem trenutku završetka među svim operacijama.

2.2.2 Poluaktivni rasporedi

Ovako opisan postupak svaku operaciju smešta u **najraniji mogući trenutak** pri zadatom redosledu. Rasporedi sa tom osobinom nazivaju se *poluaktivnim*: nijedna operacija ne može se pomeriti ulevo bez promene redosleda na nekoj mašini.

Poluaktivni rasporedi nisu najuži skup koji vredi razmatrati. Uži je skup *aktivnih* rasporeda, u kome se nijedna operacija ne može pomeriti ulevo ni uz promenu redosleda, a poznato je da taj skup sadrži bar jedno optimalno rešenje (Giffler i Thompson 1960). Postupak Giffler–Thompsona proizvodi upravo takve rasporede i time smanjuje prostor pretrage.

U ovom radu korišćen je poluaktivni dekokder, iz dva razloga. Prvo, njegova jednostavnost — jedan prolaz kroz niz, bez uslova i grananja — čini ga znatno bržim, a dekokder je funkcija koja se poziva stotinama hiljada puta po pokretanju. Drugo, skup poluaktivnih rasporeda takođe sadrži optimalno rešenje, pa izborom ovog dekokdera optimum ne postaje nedostižan, već samo prostor pretrage ostaje širi.

2.2.3 Složenost

Dekoder obavlja jedan prolaz kroz niz i za svaku operaciju konstantan broj radnji, pa je njegova složenost $O(L)$, gde je $L = \sum_j |J_j|$ ukupan broj operacija. Za instancu **ft20** to je 100 operacija po pozivu.

Kako je dekokder jedino mesto na kome se rešenje vrednuje, broj njegovih poziva korišćen je kao mera uložnog rada pri poređenju metoda, na način opisan u odeljku o metodologiji.

2.2.4 Primer

Za instancu iz odeljka o kodiranju i niz (J_1, J_2, J_1, J_2) dekokder redom dodeljuje:

Tabela 2.2: Rad dekodera nad nizom (J_1, J_2, J_1, J_2) . Vrednost C_{max} jednaka je 7.

korak	operacija	mašina	spreman	slobodna	početak	kraj
			posao	mašina		
1	J_1 , prva	M_1	0	0	0	3
2	J_2 , prva	M_2	0	0	0	2
3	J_1 , druga	M_2	3	2	3	5
4	J_2 , druga	M_1	2	3	3	7

U trećem koraku ograničenje nameće posao — operacija čeka da se završi prva operacija istog posla. U četvrtom ga nameće mašina. Ta razlika, dakle koji je od dva argumenta funkcije $\max$ bio veći, iskorišćena je pri određivanju kritičnog puta, što je opisano u narednom odeljku.

Poglavlje 3

Eksperimentalni rezultati

Kako bismo olakšali kasniju interpretaciju rezultata, za početak ćemo dostaviti kontekstualne informacije o metodologiji usporedbe dve optimizacione metode, veličini eksperimenata i okruženju na kojima su eksperimenti pokretani.

3.1 Okruženje za eksperimentisanje

Sve eksperimentalne skripte izvršavane su na privatnom računaru, čije se najznačajnije karakteristike nalaze u sledećoj tabeli:

Tabela 3.1: Karakteristike računara na kome su izvršena sva merenja.

komponenta	vrednost
procesor	Intel Core Ultra 7 165U
frekvencija	3,71 GHz
broj jezgara	12 fizičkih / 14 logičkih
radna memorija	62,2 GB
operativni sistem	Ubuntu 24.04.4 LTS, 7.0.0-28-generic kernel, x86-64
standardna biblioteka	glibc 2.39
programski jezik	Python 3.12.3, GCC 13.3.0
egzaktni rešavač	IBM ILOG CPLEX 22.2.0.0
biblioteka za modelovanje	PuLP 3.3.2

Vredi napomenuti da su sve razvijene metaheuristike implementirane tako da se izvršavaju na **jednoj niti** bez mogućnosti paralelizacije. Egzaktni rešavač (CPLEX) pokretan je sa podrazumevanim brojem niti, dakle sa svih 14 logičkih jezgara. Zbog te razlike, metode je najpreciznije porediti na osnovu broja poziva funkcije dekodiranja, a ne po utrošenom vremenu. Dužina izvršavanja navedena je kao dodatni, orijentacioni podatak.

3.2 Test instance

Pri evaluaciji razvijenih metoda korišćeno je devet različitih test instanci. Od njih devet, osam je preuzeto iz zbirke OR-Library (Beasley 1990). Instance `ft06`, `ft10` i `ft20` potiču iz rada (Fis-

her i Thompson 1963), dok instance **1a01–1a05** dolaze iz (Lawrence 1984). Poslednja, deveta, instanca **mini3** kreirana je za potrebe ovog rada. Predstavlja relativno malu instancu čiji se optimum može pronaći algoritmom grube sile. Kao takva služila je najpre pri validaciji ispravnosti implementiranih metoda optimizacije. Pregled najvažnijih informacija o svim instancama dat je u narednoj tabeli.

Tabela 3.2: Test instance sa dimenzijama, donjim granicama i objavljenim optimalnim vrednostima.

instanca	$n \times m$	operacija	donja granica	optimum	izvor optimuma
mini3	3×3	9	10	11	iscrpna pretraga
ft06	6×6	36	47	55	(Fisher i Thompson 1963)
1a01	10×5	50	666	666	donja granica
1a02	10×5	50	635	655	(Lawrence 1984)
1a03	10×5	50	588	597	(Lawrence 1984)
1a04	10×5	50	537	590	(Lawrence 1984)
1a05	10×5	50	593	593	donja granica
ft10	10×10	100	655	930	(Fisher i Thompson 1963)
ft20	20×5	100	1119	1165	(Fisher i Thompson 1963)

Donja granica u tabeli 3.2 predstavlja veću od dve trivijalne granice: (1) najdužeg posla i (2) najopterećenije mašine. Način računanja detaljno je opisan u odeljku o donjim granicama. Kod instanci **1a01** i **1a05** ta granica se poklapa sa optimumom, pa svako rešenje koje je dostigne ujedno i **dokazuje** svoju optimalnost, bez potrebe za egzaktnim rešavačem.

Instance su izabrane tako da pokriju što širi opseg težine. Konkretno, **mini3** i **ft06** sve razvijene metode rešavaju do optimuma, pa služe kao provera ispravnosti. Instance **1a01–1a05** razdvajaju metode po pouzdanosti. **ft10** i **ft20** su, uprkos skromnim dimenzijama, poznato teške. Instanca **ft10** je ostala nerešena punih dvadeset šest godina nakon objavljivanja (Jain i Meeran 1999).

3.3 Metodologija

Za početak je potrebno objasniti način izbora veličine eksperimenta. Bilo je potrebno održati balans između validnosti rezultata i razumne upotrebe (pre svega vremenskih) resursa. Sa jedne strane, ne možemo dopustiti da eksperimente izvršavamo na premalom uzorku i time dovedemo u pitanje opštost zaključaka, dok je sa druge strane potrebno da se eksperimenti izvrše u razumnom vremenskom roku.

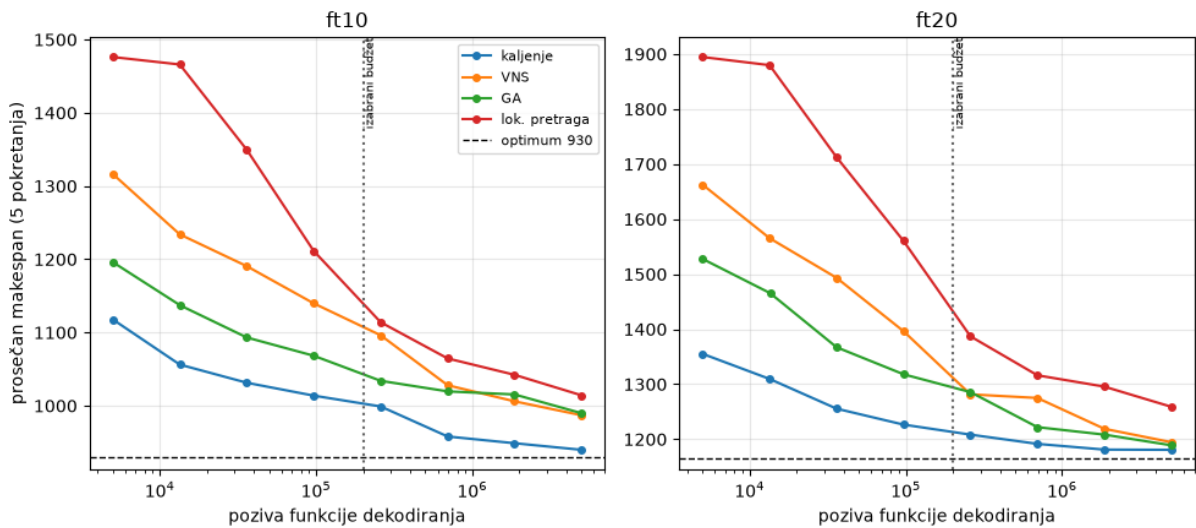
3.3.1 Budžet umesto vremena

Kao što je ranije navedeno, zbog nedostatka konkurentnosti u implementacijama algoritama, efikasnost različitih optimizacionih metoda nije pogodno upoređivati sa egzaktnim rešavačem na osnovu vremena izvršavanja. Kod međusobnog poređenja situacija je nešto bolja, ali se pokazalo

veoma izazovno unapred dozvoliti funkciji određeni vremenski okvir¹, pa je vreme izvršavanja zauzelo ulogu *retrospektivne mere*, dok su sami algoritmi unapred ograničeni brojem poziva funkcije dekodiranja.

Primetićemo ipak, a što će biti detaljnije obrađeno kasnije, da ovaj izbor normalizacije povlači sa sobom pristrasnost prema metodama koje zahtevaju manji broj poziva funkcije cilja po koraku, što im omogućava da dosegnu nešto dalje od metoda koje nemaju tu osobinu.

Svakom algoritmu je unapred data gornja granica broja poziva ove funkcije i taj broj nazivamo *budžetom*. Kao što možemo videti na slici 3.1, prosečan rezultat nastavlja blago da se popravlja i nakon izabrane granice. Međutim, poredak metoda ostaje nepromenjen a preciznije apsolutne vrednost zahtevaju nesrazmerno veći budžet. S tim na umu, izabrani budžet predstavlja kompromis između kvaliteta rešenja i ukupnog trajanje eksperimenta. Primećujemo da se vreme izvršavanja optimizacionog metoda razlikuje i pri istom budžetu, a razlog tome nalazi se u specifičnostima samih algoritama, gde se vreme „rasipa“ na pozive drugih pomoćnih metoda.



Slika 3.1: Prosečna vrednost funkcije cilja u zavisnosti od dodeljenog budžeta, za dve najteže instance. Obe ose prikazane su za pet nezavisnih pokretanja po tački. Uspravna isprekidana linija označava budžet usvojen u ostatku rada.

3.3.2 Broj ponavljanja

Po uzoru na literaturu o metaheuristikama, usvojen je izbor od **30 nezavisnih pokretanja** svake metode nad svakom instancom. Uz devet instanci i četiri različite metode dolazimo do ukupno **1080 pokretanja**. Ovo predstavlja dovoljno veliki uzorak za smisleni prikaz različitih osobina dobijenih rezultata, najpre: prosek, standardnu devijaciju, procenat dostizanja optimuma i slične.

3.3.3 Stohastičnost i reproducibilnost

Svako pokretanje optimizacionog problema u sebi sadrži stohastičnost u vidu **nasumičnog izbora početnog rešenja**. Pored toga, sve četiri metode na stohastičnost nailaze i u drugim

¹Pre svega izazovnost se ogleda u činjenici da isti vremenski okvir na istoj mašini u različitim metodama podrazumeva različit broj koraka.

delovima svoje implementacije, što nas navodi da je potrebno detaljnije obraditi pitanje **reproducibilnosti** rezultata.

Konkretno, postavljanjem iste početne vrednosti za generator pseudonasumičnih brojeva (engl. *seed*) omogućena je potpuna reproducibilnost svih dobijenih rezultata. Ponovno pokretanje priloženog koda dovodi do identičnih rezultata. Takođe *seed* vrednost sačuvana je u priloženim datotekama pa je moguće reprodukovati, ne samo eksperiment, već i jedno zasebno pokretanje optimizacione metode.

3.4 Rezultati testiranja

Dobijeni rezultati prikazani su kroz tri naredne tabele. U prvoj tabeli nalazi se **najbolje pronađeno rešenje** kroz **svih 30 pokretanja**. U drugoj tabeli vidimo prosek i standardnu devijaciju, dok treća tabela nosi informaciju o broju pokretanja u kojima je optimum dostignut.

U prvoj tabeli jasno je izražena informacija o mogućnosti (potencijalu) metode da pronađe najbolje rešenje. Kroz drugu i treću tabelu se ogleda pouzdanost različitih metoda pri višestrukoj upotrebi.

Tabela 3.3: Najbolje pronađeno rešenje kroz 30 pokretanja. Podebljane vrednosti poklapaju se sa objavljenim optimumom.

instanca	optimum	kaljenje	VNS	GA	lok. pretraga
mini3	11	11	11	11	11
ft06	55	55	55	55	55
la01	666	666	666	666	666
la02	655	655	655	655	663
la03	597	597	597	604	597
la04	590	590	590	590	594
la05	593	593	593	593	593
ft10	930	937	971	961	1002
ft20	1165	1173	1192	1180	1249

Tabela 3.4: Prosečna vrednost funkcije cilja kroz 30 pokretanja, sa standardnom devijacijom.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0
ft06	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0
la01	666,0 ± 0,0	666,0 ± 0,0	670,9 ± 8,9	666,0 ± 0,0
la02	659,0 ± 5,7	660,7 ± 6,6	669,9 ± 9,4	674,5 ± 9,0
la03	606,2 ± 7,4	607,6 ± 7,0	625,6 ± 14,2	623,9 ± 9,9
la04	591,2 ± 2,0	594,1 ± 4,2	606,0 ± 14,2	607,5 ± 7,3
la05	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0
ft10	965,2 ± 16,3	1023,8 ± 24,7	1012,1 ± 30,9	1059,6 ± 30,0
ft20	1183,8 ± 11,1	1245,8 ± 29,9	1221,0 ± 26,6	1320,1 ± 40,9

Tabela 3.5: Broj pokretanja, od ukupno 30, u kojima je dostignut poznati optimum.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	30	30	30	30
ft06	30	30	30	30
1a01	30	30	20	30
1a02	19	16	2	0
1a03	5	3	0	1
1a04	21	11	4	0
1a05	30	30	30	30
ft10	0	0	0	0
ft20	0	0	0	0

Tabela 3.3 pokazuje da instance **mini3**, **ft06** i **1a05** sve tri četiri rešavaju do optimuma. Standardna devijacija na njima je nula (tabela 3.4), pa te instance ne razdvajaju metode i služe isključivo kao provera ispravnosti implementacije.

Poređenje metoda isključivo po najboljem pronađenom rešenju prikriva stvarnu informaciju o upotrebljivosti metode. Na primer, pri rešavanju instance **1a03** sve metode, osim genetskog algoritma, dostižu optimum. Ipak, tabela 3.5 pokazuje da ga lokalna pretraga pronalazi **u samo jednom od trideset pokretanja**, dok simulirano kaljenje to uspeva pet puta. Zbog toga je pri poređenju metoda potrebno uzeti u obzir i njihovo generalno ponašanje. Nepouzdanost je ograničiti se samo na najbolje primere.

Na instancama **1a02**, **1a03** i **1a04** razlike su najizraženije. Kaljenje dostiže optimum redom 19, 5 i 21 put, promenljive okoline 16, 3 i 11 puta, dok lokalna pretraga ne uspeva gotovo nikada.

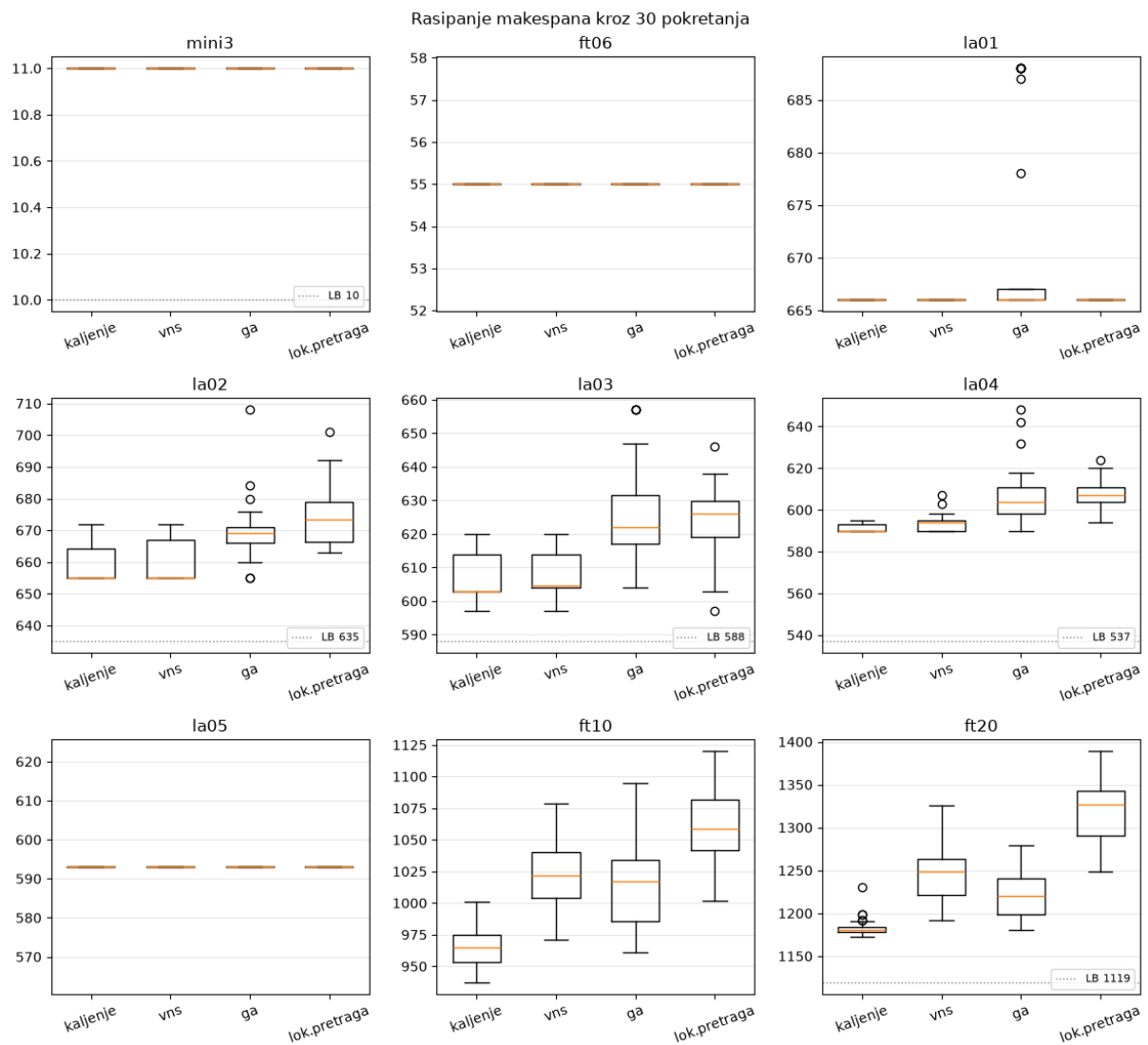
Instanca **1a03** se u rezultatima izdvaja. Naime, iako je istih dimenzija kao **1a02** i **1a04**, sve metode je rešavaju znatno teže. Uzrok nije utvrđen u okviru ovog rada.

Na najvećim instancama, **ft10** i **ft20**, nijedna metoda ne dostiže objavljeni optimum. Ipak, simulirano kaljenje je i tu najbliže, sa prosečnim odstupanjem od 3,8% odnosno 1,6%.

Posmatrano na svim instancama, prosečno odstupanje od optimuma iznosi 0,86% za kaljenje, 2,26% za promenljive okoline, 2,68% za genetski algoritam i 4,19% za lokalnu pretragu sa ponovnim pokretanjem. Poredak je isti u sve tri tabele, što ga čini pouzdanim zaključkom.

3.5 Egzaktno rešavanje

Radi provere ispravnosti razvijenih metoda i utvrđivanja stvarnih optimuma, problem je rešen i egzaktno, celobrojn timer linearnim programiranjem. Korišćena je disjunktivna formulacija (Mann 1960), opisana u odeljku o celobrojn timer modelu, dok je model rešen putem rešavača CPLEX. Pritom, svakoo pokretanje vremenski je unapred ograničeno na jedan sat i zahtevana je stroga optimalnost. Rezultati egzaktnog rešavanja dati su u sledećoj tabeli:



Slika 3.2: Rasipanje vrednosti funkcije cilja kroz 30 pokretanja, po instanci i metodi. Kutija obuhvata srednjih 50% rezultata, linija u njoj je medijana, a kružići označavaju odudarajuće vrednosti (*engl. outliers*).

Tabela 3.6: Rezultati egzaktnog rešavanja. Podebljane vrednosti su dokazano optimalne.

instanca	promenljivih	ograničenja	rešenje	donja granica	odstupanje	vreme [s]
mini3	19	27	11	11	0	0,1
ft06	127	216	55	55	0	0,3
la01	276	500	666	666	0	5,2
la02	276	500	655	655	0	3,0
la03	276	500	597	597	0	2,7
la04	276	500	590	590	0	1,6
la05	276	500	593	593	0	9,8
ft10	551	1000	930	930	0	50,8
ft20	1051	2000	1182	657	44,4 %	> 3600

Osam od devet instanci rešeno je do dokazane optimalnosti i sve dobijene vrednosti poklapaju se sa objavljenim optimumima iz tabele 3.2. Ovime je nezavisno potvrđeno da implementirani dekodier ne proizvodi neizvodljive rasporede.

Primetimo da vreme rešavanja raste izrazito brzo sa veličinom instance: 0,3 sekunde za **ft06**, oko 3 sekunde za instance **la01–la05**, 50,8 sekundi za **ft10**, dok **ft20** nije rešen ni nakon sat vremena. Upravo ovde vidimo najveću vrednost metaheuristika.

Posvetimo na kratko pažnju instanci **ft20**. Nakon sat vremena rešavač je pronašao rešenje vrednosti 1182 i dokazao donju granicu 657. U pitanju je razilaženje od preko 44%. Poređenja radi, simulirano kaljenje je u istom okruženju pronašlo rešenje vrednosti 1173 za oko pet sekundi. Rezultat je da je primenom metode optimizacije putem metaheuristika pronađeno **bolje rešenje u vremenu manjem za tri reda veličine**.

Pored toga, jednostavna donja granica opisana u odeljku o donjim granicama iznosi za **ft20** **1119**, što je znatno jače ograničenje od granice 657 koju je rešavač dokazao za sat vremena. Kombinovanjem te granice sa najboljim pronađenim rešenjem dobija se interval [1119, 1173], odnosno raskol od 4,6% umesto 44,37%.

3.6 Poređenje sa rezultatima iz literature

U cilju procene realne upotrebljivosti razvijenih metoda, njihovi rezultati upoređeni su sa optimalnim vrednostima objavljenim u literaturi. Za instance iz zbirke OR-Library te vrednosti su odavno poznate i dosegnute su specijalizovanim metodama, između ostalih egzaktnim algoritmi-
ma granjanja i ograđivanja, i tabu pretragom prilagođenom strukturi problema.

Na sedam od devet instanci simulirano kaljenje dostiže objavljeni optimum. Na preostale dve, **ft10** i **ft20**, odstupanje iznosi 0,75% odnosno 0,69%.

Tabela 3.7: Poređenje najboljih pronađenih rešenja sa objavljenim optimumima. Podebljane vrednosti poklapaju se sa optimumom.

instanca	optimum	najbolje (kaljenje)	odstupanje	ILP	odstupanje
mini3	11	11	0	11	0
ft06	55	55	0	55	0

instanca	optimum	najbolje (kaljenje)	odstupanje	ILP	odstupanje
1a01	666	666	0	666	0
1a02	655	655	0	655	0
1a03	597	597	0	597	0
1a04	590	590	0	590	0
1a05	593	593	0	593	0
ft10	930	937	0,75 %	930	0
ft20	1165	1173	0,69 %	1182	1,46 %

Na osnovu priloženih rezultata jasno se vidi da razvijene metode ipak **ne nadmašuju rezultate iz literature**. Objavljene vrednosti za instance **ft10** i **ft20** dostignute su metodima koji pri optimizaciji koriste strukturu samog problema (Nowicki i Smutnicki 1996), dok su naše implementirane metode opšte prirode i oslanjaju se na pronalaženje jednostavne okoline. Ovakav ishod je očekivan i u skladu sa obimom rada.

Vredi, međutim, još jednom osvrnuti se na odnos utrošenog vremena. Rešenje vrednosti 1173 za instancu **ft20** simulirano kaljenje pronalazi za oko pet sekundi, dok egzaktni rešavač ni nakon sat vremena ne uspeva da pronađe bolje od 1182. Za praktičnu primenu, u kojoj se raspored često mora dobiti u kratkom roku, odstupanje ispod jednog procenta uz vreme izvršavanja od nekoliko sekundi predstavlja upotrebljiv rezultat.

3.7 Diskusija

Prilokom razvoja i merenja uočeno je nekoliko pojava koje se ne vide iz zbirnih tabela, a koje su bitno uticale na konačan izbor metoda i njihovih parametara.

3.7.1 Uža okolina ne donosi bolji rezultat

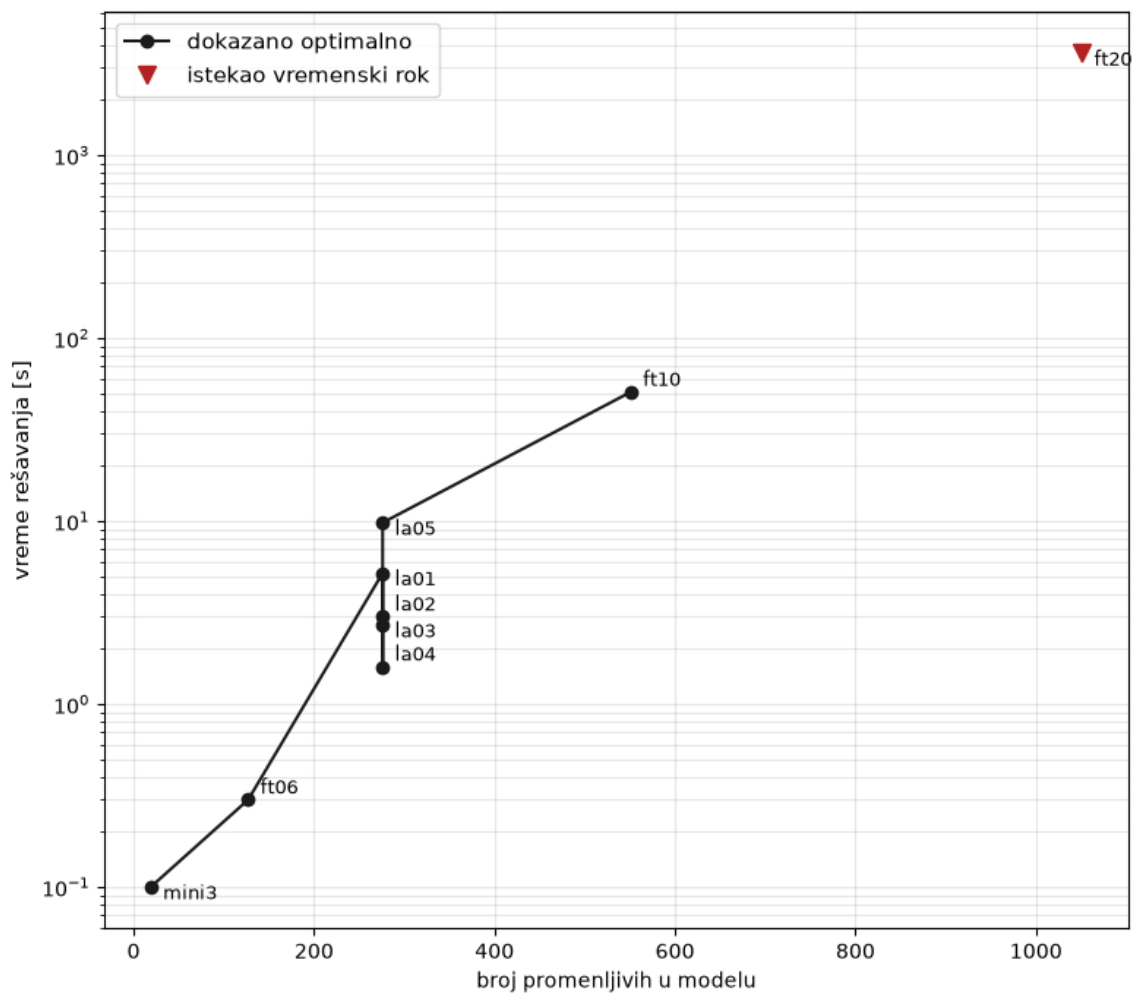
Okolina zasnovana na kritičnom putu, u literaturi poznata kao N_1 (Laarhoven, Aarts, i Lenstra 1992), razmatra samo zamene susednih operacija na kritičnom putu koje se izvršavaju na istoj mašini. Time se broj kandidata po koraku drastično smanjuje. Na primer, sa 540 na 6,7² kod instance **ft06**, sa 4500 na 17,5 kod **ft10** i sa 4750 na 31,6 kod **ft20**, dakle između 81 i 259 puta.³

Uprkos tome, pri izjednačenom broju poziva funkcije dekodiranja ta okolina daje **lošiji** rezultat od proste zamene dva elementa. Kod višestruko pokrenute lokalne pretrage na instanci **ft10** prosek je 1044,8 naspram 1032,4, a kod simuliranog kaljenja 980,6 naspram 950,0. Na instanci **ft20** razlika je je najizraženija: 1263,8 naspram 1180,4.

Postoje dva glavna uzroka zbog čega se pojavljuju ovakvi rezultati. Prvi razlog nalazi se upravo u dizajnu samog eksperimenta, odnosno u odluci da upoređivanje metoda normalizujemo na osnovu budžeta. Naime, iako se pokazalo efektno pri usporedbi metoda optimizacije u njihovom osnovnom obliku, pri pronalaženju N_1 poziva se funkcija dekodiranja, što samo po sebi umanjuje budžet. Drugo, uža okolina daje plići pojam lokalnog optimuma, što navodi pretragu na ranije zaustavljanje i to na lošijem mestu. Kod simuliranog kaljenja postoji i treći razlog. Naime,

²Date su prosečne vrednosti broja kandidata

³Ova merenja sprovedena su naknadno, s obzirom na neobećavajuće rezultate



Slika 3.3: Vreme rešavanja celobrojnog modela u zavisnosti od broja promenljivih. Vertikalna osa je logaritamska. Instanca **ft20** nije rešena u zadatom roku od sat vremena, pa njena vrednost predstavlja donju procenu potrebnog vremena.

ograničavanjem izbora na kritični put oduzimaju se neutralni potezi, koji ne menjaju vrednost funkcije cilja odmah, ali otvaraju put kasnijim poboljšanjima.

U literaturi N_1 i srodne okoline daju vrlo dobre rezultate, ali u sprezi sa tabu listom i inkrementalnom procenom vrednosti (Nowicki i Smutnicki 1996), čime se oba navedena nedostatka uklanjaju. Bez tih dopuna, sama okolina ispostavlja se nedovoljnom.

3.7.2 Složenija metoda nije nužno bolja

Metoda promenljivih okolina koristi lokalnu pretragu kao potprogram i nad njom gradi mehanizam izlaska iz lokalnog optimuma. Očekivano bi bilo da nadmaši simulirano kaljenje, koje je konceptualno jednostavnije. Merenja pokazuju suprotno: pri istom budžetu prosečno odstupanje iznosi 2,26% za promenljive okoline naspram 0,86% za kaljenje.

Razlog je u ceni jednog koraka. Jedan silazak lokalne pretrage na instanci `ft10` troši oko 40 000 poziva funkcije cilja, pa pri budžetu od 200 000 poziva metoda stigne da izvede svega nekoliko silazaka. Simulirano kaljenje u istom budžetu razmotri 200 000 pojedinačnih poteza. Pri ovako postavljenom poređenju prednost ima metoda sa jeftinijim korakom.

3.7.3 Odnos prema egzaktnom rešavanju

Rezultati na instanci `ft20` pokazuju granicu primenljivosti egzaktnog pristupa. Rešavač je za sat vremena pronašao rešenje vrednosti 1182 i dokazao donju granicu 657, dok je simulirano kaljenje za oko pet sekundi pronašlo rešenje vrednosti 1173. Uz to je jednostavna donja granica iz odeljka o donjim granicama, izračunata u zanemarljivom vremenu, iznosila 1119, što je znatno više od granice koju je rešavač dokazao.

Kombinovanjem sopstvenih rezultata, dakle donje granice 1119 i najboljeg pronađenog rešenja 1173, dobija se procena optimuma sa ostupanjem od 4,6%, umesto 44,37% koliko je iznosilo odstupanje egzaktnog rešavača.

Poglavlje 4

Zaključak

Poglavlje 5

Literatura

- Beasley, J. E. 1990. „OR-Library: Distributing Test Problems by Electronic Mail“. *Journal of the Operational Research Society* 41 (11): 1069–72.
- Bierwirth, Christian. 1995. „A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms“. *OR Spektrum* 17 (2–3): 87–92.
- Fisher, H., i G. L. Thompson. 1963. „Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules“. U *Industrial Scheduling*, uredio J. F. Muth i G. L. Thompson, 225–51. Englewood Cliffs, New Jersey: Prentice Hall.
- Giffler, B., i G. L. Thompson. 1960. „Algorithms for Solving Production-Scheduling Problems“. *Operations Research* 8 (4): 487–503.
- Jain, A. S., i S. Meeran. 1999. „Deterministic Job-Shop Scheduling: Past, Present and Future“. *European Journal of Operational Research* 113 (2): 390–434.
- Laarhoven, Peter J. M. van, Emile H. L. Aarts, i Jan Karel Lenstra. 1992. „Job Shop Scheduling by Simulated Annealing“. *Operations Research* 40 (1): 113–25.
- Lawrence, S. 1984. „Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques (Supplement)“. Pittsburgh, Pennsylvania: Graduate School of Industrial Administration, Carnegie-Mellon University.
- Manne, Alan S. 1960. „On the Job-Shop Scheduling Problem“. *Operations Research* 8 (2): 219–23.
- Nowicki, Eugeniusz, i Czeslaw Smutnicki. 1996. „A Fast Taboo Search Algorithm for the Job Shop Problem“. *Management Science* 42 (6): 797–813.